Step 1: Install and Configure Google App Engine on Ubuntu

Prerequisites

- Ubuntu (e.g., 20.04 or later recommended as of April 2025).
- Python 3.9 installed (Google App Engine supports Python 3 in the standard environment).
- Internet is Mandatory
- A Google Cloud account.

Installation Steps

1. **Update Ubuntu Packages** Open a terminal and ensure your system is up-to-date:
   run on terminal
   sudo apt update && sudo apt upgrade -y
2. **Install Python and Pip** Ubuntu typically comes with Python pre-installed. Verify the version and install pip if needed:
   run on terminal
   python3 –version
   sudo apt install -y python3.9 python3.9-dev python3.9-distutils
   sudo apt install python3-pip
   Add both versions to alternatives
   sudo update-alternatives --install /usr/bin/python3 python3 /usr/bin/python3.8 1
   sudo update-alternatives --install /usr/bin/python3 python3 /usr/bin/python3.9 2
   Choose the default version
   sudo update-alternatives --config python3
   Install Curl
   sudo apt-get install curl
3. **Install Google Cloud SDK** The Google Cloud SDK includes the tools needed to work with Google App Engine. Follow these steps:
   - Add the Google Cloud SDK repository:
     run on terminal
     echo "deb [signed-by=/usr/share/keyrings/cloud.google.gpg] https://packages.cloud.google.com/apt cloud-sdk main" | sudo tee -a /etc/apt/sources.list.d/google-cloud-sdk.list

- Import the Google Cloud public key:

  run on terminal

  curl https://packages.cloud.google.com/apt/doc/apt-key.gpg | sudo apt-key --keyring /usr/share/keyrings/cloud.google.gpg add -
- Update and install the SDK:

  run on terminal

  sudo apt update

  sudo apt install google-cloud-sdk -y

4. **Install App Engine Python Component** Install the App Engine extension for Python:

   run on terminal

   sudo apt install google-cloud-sdk-app-engine-python -y
5. **Initialize Google Cloud SDK** Authenticate and configure the SDK:

   run on terminal

   gcloud init

   - Follow the prompts to log in with your Google account via a browser.
   - Select or create a Google Cloud project better to select your oldest projet.
   - Choose a default region (e.g., asia-south1).
6. **Verify Installation** Check the installed components:

   run on terminal

   gcloud components list

   Ensure app-engine-python is marked as installed.

Step 2: Create a "Hello World" Program in Python

Directory Setup

1. Create a project directory:

   run on terminal

   mkdir hello-world-app

   cd hello-world-app
2. Set up a virtual environment (optional but recommended):

run on terminal

sudo apt install python3.9-venv

python3 -m venv venv

source venv/bin/activate

Create Required Files

You'll need three main files: app.yaml, main.py, and requirements.txt.

1. **Create app.yaml** This file configures the App Engine environment:

```
runtime: python39
entrypoint: gunicorn -b :$PORT main:app
```

   - Save this as app.yaml in the hello-world-app directory.

2. **Create main.py** This is the Python code for the "Hello World" app using Flask:

```
from flask import Flask

app = Flask(__name__)

@app.route('/')
def hello():
    return 'Hello, World!'

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=8080)
```

   - Save this as main.py.

3. **Create requirements.txt** List the dependencies:

```
Flask==2.3.3
gunicorn==20.1.0
```

   - Save this as requirements.txt.

4. **Install Dependencies Locally** With the virtual environment activated:
   run on terminal

   pip install -r requirements.txt

Step 3: Test Locally

1. Run the app locally using the development server:
   run on terminal

   sudo apt-get install google-cloud-cli-app-engine-python google-cloud-cli-datastore-emulator dev_appserver.py app.yaml
2. Open a browser and visit http://localhost:8080. You should see "Hello, World!".
3. Stop the server with Ctrl+C.

Step 4: Deploy to Google App Engine

1. Deploy the app:
   run on terminal

   gcloud app deploy app.yaml

   - Confirm the deployment when prompted.
2. Once deployed, get the URL:
   run on terminal

   gcloud app browse

Explanation of Files and Code

1. app.yaml

- **Purpose**: Configuration file for Google App Engine.
- **Contents**:
  - runtime: python39: Specifies Python 3.9 as the runtime environment (use a supported version as of April 2025; check Google Cloud docs for the latest).
  - entrypoint: gunicorn -b :
    $PORT main:app: Defines how the app starts.

gunicorn is a WSGI server,
-b :$PORT binds it to the port assigned by App Engine
(stored in the PORT environment variable), and
main:app points to the app object in main.py.

- **Role**: Tells App Engine how to run your app and what environment to use.

2. main.py

- **Purpose**: The application code that defines the web server logic.
- **Code Breakdown**:
  - from flask import Flask: Imports the Flask web framework.
  - app = Flask(__name__): Creates a Flask application instance.
    __name__ is a Python built-in variable representing the current module.
  - @app.route('/'): Decorator that maps the root URL (/) to the hello function.
  - def hello(): return 'Hello, World!': Defines a function that returns the "Hello, World!" string when the root URL is accessed.
  - if __name__ == '__main__': app.run(...): Runs the app locally on port 8080 if executed directly (not used in App Engine; included for local testing).
- **Role**: Handles HTTP requests and serves the "Hello, World!" response.

3. requirements.txt

- **Purpose**: Lists Python dependencies for the app.
- **Contents**:
  - Flask==2.3.3: Specifies the Flask version (a lightweight web framework).
  - gunicorn==20.1.0: Specifies the Gunicorn version (a WSGI server for running Python web apps in production).
- **Role**: Ensures App Engine installs these libraries during deployment using pip.

How It Works

- **Local Testing**: dev_appserver.py simulates the App Engine environment on your machine, allowing you to test before deployment.
- **Deployment**: gcloud app deploy uploads your files to Google Cloud, builds a container, and runs it on App Engine. The entrypoint in app.yaml starts Gunicorn, which serves your Flask app.

- **Access**: After deployment, your app is accessible globally via the appspot.com URL.